Classical Ruge–Stüben Algebraic Multigrid: Strength of Connection and Standard Interpolation

1 Introduction

Algebraic multigrid (AMG) methods solve sparse linear systems $Ax = b$ by constructing a hierarchy of progressively coarser problems without requiring geometric grid information. The classical AMG method of Ruge and Stüben [**?**] achieves this through three key components:

1. **Strength of connection** — identifying which matrix entries represent important couplings between unknowns.
2. **Coarse/fine (C/F) splitting** — partitioning unknowns into coarse points (retained on the next level) and fine points (eliminated by interpolation).
3. **Standard interpolation** — constructing the prolongation operator P that maps coarse-grid corrections to the fine grid.

Given P, the restriction operator is $R = P^T$ and the coarse-grid operator is formed by the Galerkin product $A_c = P^T AP$.

Throughout, we assume $A \in \mathbb{R}^{n\times n}$ is an M-matrix: $a_{ii} > 0$, $a_{ij} \leq 0$ for $j \neq i$. We denote the set of neighbors of point i (off-diagonal entries in row i) by $\mathcal{N}_i = \{j \neq i : a_{ij} \neq 0\}$.

2 Strength of Connection

The fundamental observation underlying AMG is that relaxation methods (e.g., Gauss–Seidel) effectively reduce oscillatory error components but leave *algebraically smooth* error — error that satisfies $Ae \approx 0$ pointwise — largely unchanged [**?**, **?**]. For the i-th equation,

$$a_{ii}\, e_i + \sum_{j\in\mathcal{N}_i} a_{ij}\, e_j \approx 0$$

implies that e_i depends most strongly on neighbors j for which $|a_{ij}|$ is large relative to the other couplings in the row.

[Strong connection] Given a threshold parameter $\theta \in (0, 1]$, point j is a **strong connection** of point i if

$$-a_{ij} \;\geq\; \theta \cdot \max_{k\neq i}\bigl(-a_{ik}\bigr). \tag{1}$$

The set of strong connections of i is denoted S_i. We also define the **strong transpose**:

$$S_i^T = \{j : i \in S_j\}$$

which is the set of points that i strongly influences.

Typical values are $\theta = 0.25$ for two-dimensional problems and $\theta = 0.5$ for three-dimensional problems [**?**]. For an M-matrix, $-a_{ij} \geq 0$ for all $j \neq i$, so the criterion (**??**) selects off-diagonal entries whose magnitude is at least a fraction θ of the row maximum.

The strength matrix S is stored in compressed sparse row (CSR) format containing only the strong off-diagonal entries. Construction requires three passes over A: (i) compute the row-wise maximum, (ii) count strong entries per row, (iii) fill the CSR arrays. The total cost is $O(\mathrm{nnz}(A))$.

3 C/F Splitting

The coarsening step partitions $\{1, \ldots, n\}$ into a set of **coarse points** $\mathcal{C}$ and **fine points** $\mathcal{F}$. The splitting should satisfy two heuristic principles [**?**]:

(H1) For each F-point i, every point $j \in S_i$ is either a C-point or is strongly connected to a point in $\mathcal{C}_i = S_i \cap \mathcal{C}$.

(H2) The C-points form a maximal independent set with respect to the strong connections (no two C-points are strongly connected).

Heuristic (H1) ensures that the interpolation formula (Section **??**) can account for all important couplings of each F-point. Heuristic (H2) keeps the coarse grid as small as possible. In practice (H2) is treated as a soft guideline; the coarsening ratio $n/|\mathcal{C}|$ is typically between 2 and 3 for structured grids.

3.1 First Pass: Greedy Independent Set

The algorithm uses the *influence measure*

$$\lambda_i = \big|\{j \in S_i^T : j \text{ is undecided}\}\big|, \tag{2}$$

which counts how many undecided points are strongly influenced by i. Computing λ requires the transpose S^T.

Algorithm 1 Classical RS Coarsening — First Pass

1: Initialize $\lambda_i \leftarrow |S_i^T|$ and $\mathrm{cf}[i] \leftarrow$ UNDECIDED for all i
2: **while** undecided points remain **do**
3: Select $i^* = \arg\max_i \{\lambda_i : \mathrm{cf}[i] = \text{UNDECIDED}\}$
4: $\mathrm{cf}[i^*] \leftarrow C$ ▷ i^* becomes a coarse point
5: **for** each $j \in S_{i^*}^T$ with $\mathrm{cf}[j] = \text{UNDECIDED}$ **do**
6: $\mathrm{cf}[j] \leftarrow F$ ▷ influenced points become fine
7: **for** each $k \in S_j$ with $\mathrm{cf}[k] = \text{UNDECIDED}$, $k \neq i^*$ **do**
8: $\lambda_k \leftarrow \lambda_k + 1$ ▷ k gains value: j is now F and needs a C-neighbor
9: **end for**
10: **end for**
11: **end while**

The increment on line 8 is the key mechanism: when j becomes an F-point, it will need C-points nearby for interpolation. The undecided points that j depends on ($k \in S_j$) become more attractive as future C-points, so their λ increases. This biases subsequent selections toward fulfilling heuristic (H1).

3.2 Second Pass

After the first pass, rare configurations may leave an F-point with no strong C-neighbor. The second pass promotes any such F-point to C:

$$\text{If } i \in \mathcal{F} \text{ and } S_i \cap \mathcal{C} = \emptyset, \quad \text{then } i \to \mathcal{C}.$$

3.3 Efficient Implementation via Bucket Structure

A naive implementation scans all n points to find $\arg\max \lambda$ at each step, giving $O(n^2)$ total cost. This is replaced by a *bucket structure* [**?**, **?**] that maintains the priority queue in $O(1)$ per operation.

Data structure. Maintain an array of doubly-linked lists indexed by λ value:

- `bucket_head`$[k]$: head of the list of points with $\lambda = k$, for $k = 0, 1, \ldots, B-1$.
- `next`$[i]$, `prev`$[i]$: forward and backward pointers for point i within its bucket list (-1 as sentinel).
- `top_bucket`: index of the highest non-empty bucket.

The number of buckets B is bounded by the initial maximum λ plus the maximum row length of S (since λ values only increase via the increment on line 8 of Algorithm **??**, and each point's λ is incremented at most once per F-point creation in its neighborhood).

Operations. All three priority-queue operations run in $O(1)$ amortized time:

1. FindMax: Return `bucket_head`[`top_bucket`]. If the bucket is empty, decrement `top_bucket` until a non-empty bucket is found. Over the entire algorithm, `top_bucket` decreases at most B times total, so the amortized cost is $O(1)$.
2. Remove(i): Unlink i from its doubly-linked list. Constant time.
3. Move($i, \lambda_{\text{old}} \to \lambda_{\text{new}}$): Remove from bucket λ_{old}, insert at the head of bucket λ_{new}, update `top_bucket` if $\lambda_{\text{new}} >$ `top_bucket`. Constant time.

When a point is selected as C or marked as F, it is removed from its bucket and not reinserted. When λ_k is incremented (line 8), point k is moved from bucket λ_k to bucket $\lambda_k + 1$.

Total complexity. Each point is inserted, removed, and possibly moved a bounded number of times. Each edge of S is traversed at most twice (once via S^T when the C-point is selected, once via S when the F-point increments its neighbors). The total work is therefore $O(n + \text{nnz}(S))$.

4 Standard (Direct) Interpolation

The prolongation operator $P \in \mathbb{R}^{n \times n_c}$ is defined row by row. For a C-point i with coarse index $\hat{\imath}$, row i of P has a single entry:

$$P_{i,\hat{\imath}} = 1 \qquad \text{(identity injection)}.$$

For an F-point i, the weights are derived from the algebraic smoothness assumption.

4.1 Neighbor Classification

For F-point i, partition its neighbors into three sets:

Set	Definition	Role
$\mathcal{C}_i$	$\{j \in S_i : j \in \mathcal{C}\}$	Strong C-neighbors (interpolation sources)
D_i^s	$\{k \in S_i : k \in \mathcal{F}\}$	Strong F-neighbors (redistributed to $\mathcal{C}_i$)
D_i^w	$\mathcal{N}_i \setminus S_i$	Weak connections (lumped into diagonal)

4.2 Derivation

Start from the smoothness condition for row i:

$$a_{ii}\, e_i + \sum_{j \in \mathcal{C}_i} a_{ij}\, e_j + \sum_{k \in D_i^s} a_{ik}\, e_k + \sum_{m \in D_i^w} a_{im}\, e_m \;\approx\; 0. \tag{3}$$

Step 1: Lump weak connections. For $m \in D_i^w$, the connection to i is weak, so the error varies little: $e_m \approx e_i$. Substituting and collecting:

$$\hat{a}_{ii}\, e_i + \sum_{j \in \mathcal{C}_i} a_{ij}\, e_j + \sum_{k \in D_i^s} a_{ik}\, e_k \;\approx\; 0, \tag{4}$$

where the **modified diagonal** is

$$\hat{a}_{ii} = a_{ii} + \sum_{m \in D_i^w} a_{im}. \tag{5}$$

Step 2: Redistribute strong F-neighbors. For $k \in D_i^s$, approximate e_k by interpolation from the C-points in $\mathcal{C}_i$:

$$e_k \;\approx\; \sum_{j \in \mathcal{C}_i} \frac{a_{kj}}{\sum_{\ell \in \mathcal{C}_i} a_{k\ell}}\, e_j. \tag{6}$$

This distributes the contribution of F-neighbor k to the C-points in $\mathcal{C}_i$, weighted proportionally to the strength of connection from k to each C-point. If k has no connection to any point in $\mathcal{C}_i$ (the denominator in (**??**) vanishes), then a_{ik} is added to $\hat{a}_{ii}$ instead.

Step 3: Solve for interpolation weights. Substituting (**??**) into (**??**):

$$\hat{a}_{ii}\, e_i + \sum_{j \in \mathcal{C}_i} \underbrace{\left(a_{ij} + \sum_{k \in D_i^s} \frac{a_{ik}\, a_{kj}}{\sum_{\ell \in \mathcal{C}_i} a_{k\ell}} \right)}_{\text{numerator for } w_{ij}} e_j \;\approx\; 0.$$

Solving for $e_i \approx \sum_{j \in \mathcal{C}_i} w_{ij}\, e_j$ gives the interpolation weights:

$$\boxed{w_{ij} = -\frac{1}{\hat{a}_{ii}} \left(a_{ij} + \sum_{k \in D_i^s} \frac{a_{ik}\, a_{kj}}{\sum_{\ell \in \mathcal{C}_i} a_{k\ell}} \right), \qquad j \in \mathcal{C}_i.} \tag{7}$$

4.3 Properties

1. **Constant preservation.** If row i of A has zero row sum ($\sum_j a_{ij} = 0$), then $\sum_{j \in \mathcal{C}_i} w_{ij} = 1$. This holds for interior nodes of a Laplacian and ensures that the constant vector lies in the range of P.

   Proof. When $\sum_j a_{ij} = 0$, we have $a_{ii} = -\sum_{j \neq i} a_{ij}$, and consequently $\hat{a}_{ii} = -\sum_{j \in \mathcal{C}_i} a_{ij} - \sum_{k \in D_i^s} a_{ik}$. Similarly, $\sum_{\ell \in \mathcal{C}_i} a_{k\ell} = -a_{kk} - \sum_{m \notin \mathcal{C}_i} a_{km}$ for each k. Summing (**??**) over $j \in \mathcal{C}_i$ and using $\sum_{j \in \mathcal{C}_i} a_{kj} = \sum_{\ell \in \mathcal{C}_i} a_{k\ell}$ (the denominator) gives $\sum_j w_{ij} = -(\sum_{j \in \mathcal{C}_i} a_{ij} + \sum_{k \in D_i^s} a_{ik})/\hat{a}_{ii} = 1$. □

2. **Non-negative weights.** For M-matrices, $a_{ij} \leq 0$ for $j \neq i$ and $a_{ii} > 0$, so $\hat{a}_{ii} > 0$, the numerator in (**??**) is non-positive, and therefore $w_{ij} \geq 0$.

3. **Sparsity.** Each F-point row of P has at most $|S_i \cap \mathcal{C}|$ entries (the number of strong C-neighbors). The total nonzeros in P equal $|\mathcal{C}| + \sum_{i \in \mathcal{F}} |\mathcal{C}_i|$.

4.4 Complexity

For each F-point i, the dominant cost is the inner loop over row k of A for each $k \in D_i^s$, giving $O(|D_i^s| \cdot d_{\max})$ where $d_{\max}$ is the maximum row length of A. Summed over all F-points, the total cost is $O(\mathrm{nnz}(A) \cdot d_{\max})$ in the worst case, but $O(\mathrm{nnz}(A))$ for bounded-stencil matrices.

4.5 Algorithm Summary

Algorithm 2 Standard Interpolation for F-point i

```
Identify C_i, D_i^s, D_i^w from S_i and the C/F splitting
â_ii ← a_ii + Σ_{m∈D_i^w} a_im
for each j ∈ C_i do
    w_ij ← a_ij                                   ▷ direct connection
end for
for each k ∈ D_i^s do
    d ← Σ_{ℓ∈C_i} a_kℓ
    if |d| < ε then
        â_ii ← â_ii + a_ik                        ▷ no common C-points: lump to diagonal
    else
        for each ℓ ∈ C_i with a_kℓ ≠ 0 do
            w_iℓ ← w_iℓ + a_ik a_kℓ / d           ▷ redistribute proportionally
        end for
    end if
end for
for each j ∈ C_i do
    w_ij ← − w_ij / â_ii                          ▷ normalize
end for
```

5 Two-Level Cycle

With P constructed, the standard two-level correction scheme is:

1. **Pre-smooth:** Apply ν_1 sweeps of relaxation to $Ax = b$.
2. **Restrict residual:** $r_c = P^T(b - Ax)$.
3. **Coarse solve:** Solve $A_c\, e_c = r_c$ where $A_c = P^T AP$.
4. **Prolongate and correct:** $x \leftarrow x + P\, e_c$.
5. **Post-smooth:** Apply ν_2 sweeps of relaxation.

Applying the coarse solve recursively yields the V-cycle. The Galerkin product $A_c = P^T AP$ ensures that the coarse-grid equation is a variational projection of the fine-grid problem, which is essential for convergence [**?**, **?**].

6 Shared Library and Julia Interface

The solver is packaged as a shared library (`lib/libamg.so`) with an opaque-handle C API suitable for foreign function interfaces. This allows the AMG+ichol preconditioned CG solver to be called from any language that supports C interop (Julia, Python, Fortran, etc.) without linking individual object files.

6.1 Library API

The wrapper (`src/amg_wrapper.c`) exposes five functions:

Function	Description
`amg_solver_create`	Accepts 0-based CSR arrays $(n, \mathrm{nnz}, \texttt{ia}, \texttt{ja}, \texttt{val})$ plus parameters $(\theta, \texttt{min_size}, \texttt{max_levels}, \nu)$. Copies the input, symmetrizes $A \leftarrow (A + A^T)/2$, builds the AMG hierarchy, and computes the ichol(0) factor. Returns an opaque `void*` handle.
`amg_solver_solve`	Runs PCG with the AMG+ichol preconditioner. Takes the handle, right-hand side b, solution vector x (zeroed on entry), tolerance and maximum iterations. Returns the iteration count (negative if not converged).
`amg_solver_size`	Returns n, the matrix dimension.
`amg_solver_num_levels`	Returns the number of AMG levels.
`amg_solver_free`	Releases all memory associated with the handle.

Internally, the handle points to a struct holding the symmetrized `dcsr_mat` A, the `amg_data` hierarchy, and the `ichol_factor` L. The preconditioner callback reuses `amg_ichol_precond` from the core library.

6.2 Build

The `Makefile` compiles all library sources in `src/` with `-fPIC` and links them into a shared library:

```
make            # produces lib/libamg.so and test_amg
make clean      # removes objects, test_amg, and lib/libamg.so
```

The test driver `test_amg` links against `lib/libamg.so` with an `$ORIGIN/lib` rpath, so no `LD_LIBRARY_PATH` configuration is needed.

6.3 Julia Interface

The script `solve_amg.jl` calls `libamg.so` via Julia's built-in `ccall` mechanism, using only standard library modules (`SparseArrays`, `LinearAlgebra`, `Random`, `Printf`).

Usage.

```
julia solve_amg.jl <matrix_file> [threshold=0.25] [nu=1]
```

CSR conversion. Julia stores sparse matrices in compressed sparse column (CSC) format. To obtain 0-based CSR arrays for the C library, the script transposes the matrix: `SparseMatrixCSC`(A^T) produces CSC of A^T, which is identical to CSR of A. Subtracting 1 from `colptr` and `rowval` converts from Julia's 1-based to the library's 0-based indexing.

Library resolution. Julia's `ccall` requires the library path to be a compile-time constant. The script declares

```
const LIBAMG = joinpath(@__DIR__, "lib", "libamg.so")
```

so that `@__DIR__` resolves to the directory containing the script (the project root), and the library is found at `lib/libamg.so` without any environment variable configuration.

Data flow.

1. Read the COO file into a Julia sparse matrix and symmetrize.
2. Convert to 0-based CSR and call `amg_solver_create`.
3. Generate a random right-hand side $b \in \mathbb{R}^n$.
4. Call `amg_solver_solve` to fill the solution x.
5. Compute the relative residual $\|b - Ax\|/\|b\|$ in Julia as an independent verification.
6. Call `amg_solver_free` to release C-side memory.

References

[1] J. W. Ruge and K. Stüben, "Algebraic multigrid," in *Multigrid Methods* (S. F. McCormick, ed.), Frontiers in Applied Mathematics, vol. 3, SIAM, Philadelphia, 1987, pp. 73–130.

[2] K. Stüben, "A review of algebraic multigrid," *Journal of Computational and Applied Mathematics*, vol. 128, no. 1–2, pp. 281–309, 2001.

[3] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*, Academic Press, 2001.

[4] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.

[5] V. E. Henson and U. M. Yang, "BoomerAMG: A parallel algebraic multigrid solver and preconditioner," *Applied Numerical Mathematics*, vol. 41, no. 1, pp. 155–177, 2002.